

REAL ESTATE PROPERTY LISTING & MANAGEMENT SYSTEM

FULL-STACK BUILD PLAN • DATABASE • CARDINALITY • KEYS • NORMALIZATION

Implementation-ready plan for a 3-member team. This expanded version preserves the 7-page source plan while adding the complete relationship/cardinality contract, key mapping, schema traceability, normalization responsibilities, API/database dependencies, and phase-by-phase ownership.

0. Executive Decision Summary

Decision	Final choice	Why it is locked
Architecture	React/Vite → Express/Node → PostgreSQL	Clear separation of UI, API/business logic, and relational database.
Database access	Raw SQL with node-postgres (pg)	Makes PK/FK, joins, constraints, normalization and advanced SQL visible for DBMS grading.
ORM	Do not use Prisma in MVP	Avoids hiding gradeable SQL behind generated queries; repository/DAO layer keeps code maintainable.
Team split	Frontend / Backend+DB / Integration	Allows parallel work once shared contracts are frozen.
Schema source of truth	schema.sql	All table/column/key/constraint changes require all-3 approval.
API source of truth	API_CONTRACT.md or OpenAPI	Frontend mocks and backend controllers implement the same request/response shapes.
Advanced phase	Views, triggers, indexes, functions, EXPLAIN ANALYZE, optional PostGIS	Adds genuine DBMS depth after MVP is stable.

1. Pre-Coding Lock: The Contracts All 3 Members Must Approve

The original source plan explicitly requires freezing the database schema, API contract, vocabulary, authentication, errors, Git ownership, environment setup, UI system, and Definition of Done before parallel implementation begins.

Contract	What must be frozen	Owner
Database schema	Exact table names, columns, types, PKs, FKs, nullability, defaults, CHECKs, subtype rules, delete/update behavior	Backend + DB; all 3 approve
Cardinality & relationship rules	All 14 EER relationships below	Database; all 3 approve
API contract	Method, path, params, request, response, status codes, auth	Integration; all 3 approve
Vocabulary	customerId, agentId, propertyId, listingId, transactionId, status values, roles	Integration
Auth	JWT payload, Bearer header,	Backend + Integration

	expiration, customer/agent route permissions	
Errors	Single shape, e.g. {error: {code,message,details}}	Backend + Integration
Environment	DATABASE_URL, JWT_SECRET, VITE_API_URL, ports, CORS origin, seed command	Integration
UI system	Typography, spacing, radius, colors, buttons, cards, inputs, tables, responsive breakpoints	Frontend
Definition of Done	Happy path + validation + loading + empty + error + auth/role + DB transaction where needed + API test + integration smoke test	All 3

2. Final EER Relationship & Cardinality Contract

Use the team-provided handwritten relation list as the source of truth. Do not silently change these cardinalities during implementation.

#	Relationship	Cardinality	Relational consequence
1	PERSON → CUSTOMER	1 : 1	CUSTOMER carries PersonID FK
2	PERSON → AGENT	1 : 1	AGENT carries PersonID FK
3	CUSTOMER — Makes — OFFER	1 : N	CustomerID FK in OFFER
4	CUSTOMER — Owns — PROPERTY	N : N	Create OWNERSHIP junction table; composite PK
5	OFFER — For — LISTING	N : 1	ListingID FK in OFFER
6	PROPERTY — HasListing — LISTING	1 : N	PropertyID FK in LISTING
7	AGENT — Manages — LISTING	1 : N	AgentID FK in LISTING
8	PROPERTY — HasType — PROPERTY_TYPE	N : 1	PropertyTypeID FK in PROPERTY
9	LISTING — ResultsIn — TRANSACTION	1 : N	ListingID FK in TRANSACTION
10	TRANSACTION — SettledBy — PAYMENT	1 : N	TransactionID FK in PAYMENT
11	CUSTOMER — WrittenBy — REVIEW	1 : N	CustomerID FK in REVIEW
12	LISTING — ReviewedBy — REVIEW	1 : N	ListingID FK in REVIEW
13	TRANSACTION → SALE_TRANSACTION	1 : 1	Subtype PK is also FK to TRANSACTION
14	TRANSACTION → RENTAL_CONTRACT	1 : 1	Subtype PK is also FK to TRANSACTION

3. Key & Foreign-Key Mapping Contract

Relation	Primary Key	Foreign Keys	Notes
PERSON	PersonID	None	Age is derived from DateOfBirth; do not persist Age in normalized MVP

			schema.
CUSTOMER	CustomerID	PersonID → PERSON.PersonID	1:1 subtype mapping.
AGENT	AgentID	PersonID → PERSON.PersonID	1:1 subtype mapping.
PROPERTY	PropertyID	PropertyTypeID → PROPERTY_TYPE.PropertyTypeID	N:1 with PROPERTY_TYPE; ownership is represented separately through OWNERSHIP.
PROPERTY_TYPE	PropertyTypeID	None	Strong entity under the final N:1 HasType mapping.
OWNERSHIP	(CustomerID, PropertyID)	CustomerID → CUSTOMER; PropertyID → PROPERTY	N:N relationship table; attributes OwnershipShare, SinceDate belong here.
LISTING	ListingID	PropertyID → PROPERTY; AgentID → AGENT	Property and agent are dimensions of a listing.
OFFER	OfferID	CustomerID → CUSTOMER; ListingID → LISTING	One customer can make many offers; each offer targets one listing.
TRANSACTION	TransactionID	ListingID → LISTING	Add TransactionType discriminator for sale/rental enforcement.
SALE_TRANSACTION	TransactionID	TransactionID → TRANSACTION	PK/FK subtype row.
RENTAL_CONTRACT	TransactionID	TransactionID → TRANSACTION	PK/FK subtype row.
PAYMENT	PaymentID	TransactionID → TRANSACTION	Many payments can settle one transaction.
REVIEW	ReviewID	CustomerID → CUSTOMER; ListingID → LISTING	Review ties customer and listing.

4. Final Relational Schema (MVP Baseline)

PERSON(PersonID PK, FirstName, MiddleName, LastName, Email, PhoneNo, DateOfBirth)
 CUSTOMER(CustomerID PK, PersonID FK)
 AGENT(AgentID PK, PersonID FK)
 PROPERTY_TYPE(PropertyTypeID PK, TypeName)
 PROPERTY(PropertyID PK, AreaSqFt, Price, Description, PropertyTypeID FK)
 OWNERSHIP(CustomerID PK/FK, PropertyID PK/FK, OwnershipShare, SinceDate)
 LISTING(ListingID PK, PropertyID FK, AgentID FK, ListPrice, ListedDate, Status)
 OFFER(OfferID PK, CustomerID FK, ListingID FK, OfferAmount, OfferDate)
 TRANSACTION(TransactionID PK, ListingID FK, TransactionDate, TransactionType)
 SALE_TRANSACTION(TransactionID PK/FK, SalePrice, RegistrationNo)
 RENTAL_CONTRACT(TransactionID PK/FK, MonthlyRent, LeaseStartDate, LeaseEndDate)
 PAYMENT(PaymentID PK, TransactionID FK, Amount, PaymentDate, PaymentMode, ReferenceNo)
 REVIEW(ReviewID PK, CustomerID FK, ListingID FK, ReviewDate, Rating, Comment)

TransactionType is an implementation-level discriminator added to enforce the exclusive sale/rental specialization. The source build plan recommends this approach together with trigger/application enforcement.

5. Referential-Integrity & Constraint Rules

Table	Constraint	Purpose
PERSON	UNIQUE Email; optionally UNIQUE	Avoid duplicate identity/contact

	PhoneNo	records.
CUSTOMER / AGENT	PersonID UNIQUE FK	Preserve intended 1:1 subtype mapping.
PROPERTY	Price > 0; AreaSqFt > 0; PropertyTypeID NOT NULL	Valid property inventory.
OWNERSHIP	OwnershipShare > 0 AND <= 100	Valid percentage.
OWNERSHIP	(optionally) trigger total share per property <= 100	Prevent impossible ownership totals.
LISTING	ListPrice > 0; Status in agreed enum	Prevent invalid listings.
OFFER	OfferAmount > 0	Prevent invalid offers.
TRANSACTION	TransactionType IN (SALE, RENTAL)	Drive exclusive subtype logic.
SALE_TRANSACTION	RegistrationNo UNIQUE	Avoid duplicate registrations.
RENTAL_CONTRACT	MonthlyRent > 0; LeaseStartDate < LeaseEndDate	Valid lease.
PAYMENT	Amount > 0; ReferenceNo UNIQUE	Valid and traceable payment.
REVIEW	Rating BETWEEN 1 AND 5	Valid rating range.
All FKs	Define deliberate ON DELETE / ON UPDATE behavior	Prevent accidental orphaning or unintended cascades.

6. Individual Normalization Plan

Normalization is to be demonstrated relation-by-relation, not as one giant combined relation. Do not invent dependencies just to force BCNF. The final schema should preserve the EER semantics and the project should show genuine dependencies/violations where they exist.

Relation / case	Key idea	Normal forms to demonstrate	Expected action
PERSON	Age is derived from DateOfBirth	1NF → 3NF/BCNF	Remove stored Age from final schema.
OWNERSHIP	Composite key (CustomerID, PropertyID)	1NF → 2NF → 3NF → BCNF	Use as the strongest composite-key example; relationship attributes depend on the whole key.
LISTING	If AgentName is included in a pre-normalization study relation, AgentID → AgentName	1NF → 3NF	Move agent descriptive data to AGENT/PERSON; final LISTING uses AgentID only.
OFFER	If CustomerName is included in the study relation, CustomerID → CustomerName	1NF → 3NF	Keep customer identity data in PERSON/CUSTOMER.
TRANSACTION	ListingID determines property context through LISTING	3NF review	Do not redundantly store property data in TRANSACTION.
PROPERTY_TYPE	PropertyTypeID → TypeName	1NF → 3NF → BCNF	Keep type catalog separate; final PROPERTY references PropertyTypeID.
Remaining relations	Single-key determinant usually means no partial dependency	1NF → 2NF → 3NF → BCNF verification	State the test honestly; no artificial decomposition.

7. Backend + Database Build Plan

The source plan recommends PostgreSQL plus raw SQL with pg, a thin repository/DAO layer, pg-pool, .env credentials, and hand-written SQL artifacts. [NfileciteÖturn0file0ÖL54-L64Ö](#)

7.1 Database file structure

```
db/  
  schema.sql  
  seed.sql  
  constraints.sql  
  indexes.sql  
  views.sql  
  triggers.sql  
  functions.sql  
  queries.sql
```

File	MVP?	Contents
schema.sql	YES	CREATE TABLE statements in dependency order; PK/FK definitions.
seed.sql	YES	Realistic demo data that satisfies all FK and CHECK constraints.
constraints.sql	YES	Additional UNIQUE/CHECK constraints if kept separate from DDL.
indexes.sql	YES	Indexes for FK/search/sort columns; compare with EXPLAIN ANALYZE later.
views.sql	Recommended	At least one non-trivial browse/report view.
triggers.sql	Recommended	Exclusive transaction subtype enforcement and/or listing close/status automation.
functions.sql	Advanced	Stored functions once core CRUD and transaction flows work.
queries.sql	YES	Basic CRUD, joins, aggregate/report queries, CTEs/subqueries as appropriate.

7.2 Backend structure

```
backend/  
  src/  
    config/db.js  
    middleware/auth.js  
    middleware/errorHandler.js  
    middleware/roleGuard.js  
    repositories/  
      personRepo.js  
      customerRepo.js  
      agentRepo.js  
      propertyRepo.js  
      propertyTypeRepo.js  
      ownershipRepo.js  
      listingRepo.js  
      offerRepo.js  
      transactionRepo.js  
      saleTransactionRepo.js  
      rentalContractRepo.js  
      paymentRepo.js  
      reviewRepo.js
```

```

controllers/
routes/
validators/
services/
app.js
server.js
.env
package.json

```

Rule: route → controller → service → repository → parameterized SQL. Controllers should not contain SQL strings. This preserves maintainability while keeping the actual SQL visible for grading.

7.3 Authentication & authorization

Register -> PERSON + CUSTOMER/AGENT in one DB transaction

Login -> verify password hash -> issue JWT

JWT -> { personId, role, customerId | agentId }

Protected route -> auth middleware -> roleGuard -> controller

- Hash passwords with bcrypt; never store plaintext passwords.
- Use Authorization: Bearer <token> consistently.
- Backend, not frontend alone, enforces customer/agent permissions.
- Keep JWT secret and DB credentials in environment variables.

7.4 Core REST API catalogue

Resource	Endpoints	Main tables
Auth	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me	PERSON, CUSTOMER, AGENT
Property Types	GET /api/property-types; POST/PUT/DELETE if admin workflow is added	PROPERTY_TYPE
Properties	GET/POST/PUT/DELETE /api/properties; GET /api/properties/:id	PROPERTY, PROPERTY_TYPE, OWNERSHIP
Ownership	GET /api/ownerships; POST /api/ownerships; DELETE /api/ownerships/:id	OWNERSHIP
Listings	GET/POST/PUT/DELETE /api/listings; GET /api/listings/:id	LISTING, PROPERTY, AGENT, PERSON
Offers	POST /api/offers; GET /api/offers/:id; GET /api/offers?customerId=...; GET /api/offers?listingId=...; PATCH /api/offers/:id/status	OFFER, CUSTOMER, LISTING
Transactions	GET/POST /api/transactions; GET /api/transactions/:id	TRANSACTION + subtype
Payments	GET/POST /api/payments; GET /api/payments/:id	PAYMENT, TRANSACTION
Reviews	GET /api/listings/:id/reviews; POST /api/listings/:id/reviews; PUT/DELETE /api/reviews/:id	REVIEW, CUSTOMER, LISTING

8. SQL / DBMS Demonstration Requirements

- Basic CRUD queries for core entities.
- JOIN queries crossing PROPERTY, PROPERTY_TYPE, LISTING, AGENT and PERSON.

- Aggregate reports: count listings per agent, average price by type, total payment per transaction, offer counts per listing.
- At least one non-trivial view used by the browse endpoint; the source plan explicitly recommends an active-listings view. NfileciteÖturn0file0ÖL164-L175Ö
- At least one trigger: enforce exclusive sale/rental subtype or automatically close listing when a transaction is created.
- At least one real SQL transaction using BEGIN/COMMIT/ROLLBACK, especially offer-acceptance → transaction creation.
- EXPLAIN ANALYZE before/after an index on a representative browse/report query.
- Optional advanced: CTEs, window functions, stored functions, materialized view, PostGIS.

9. Frontend Build Plan

```

frontend/
src/
  api/
    axiosClient.js
    authApi.js
    propertiesApi.js
    propertyTypesApi.js
    ownershipApi.js
    listingsApi.js
    offersApi.js
    transactionsApi.js
    paymentsApi.js
    reviewsApi.js
  components/
    Navbar.jsx
    PropertyCard.jsx
    ListingCard.jsx
    SearchBar.jsx
    FilterPanel.jsx
    OfferForm.jsx
    PaymentForm.jsx
    ReviewForm.jsx
    ProtectedRoute.jsx
    LoadingState.jsx
    ErrorState.jsx
  pages/
    Home.jsx
    Login.jsx
    Register.jsx
    BrowseListings.jsx
    PropertyDetails.jsx
    CustomerDashboard.jsx
    AgentDashboard.jsx
    Offers.jsx
    Transactions.jsx
    Payments.jsx
    Reviews.jsx
    Profile.jsx
    ListingForm.jsx
  context/AuthContext.jsx
  hooks/useAuth.js
  App.jsx
  main.jsx

```

Frontend area	Required behavior	Definition of done
---------------	-------------------	--------------------

Auth	Register/login/logout; role-aware navigation	Works with mock API first and real API after integration; validation and 401 handling.
Browse	Card grid, type/price/status filters, sorting, loading/empty/error states	User can find a listing without page reload.
Property detail	Property, type, current listing, agent, reviews, offer CTA	Customer can reach offer flow; agent sees edit controls when authorized.
Customer dashboard	Owned properties, offers, transactions, reviews, payments	All panels use API-driven data and clear empty states.
Agent dashboard	Managed listings, offers received, listing edit/create, transaction/payment tools	Role-gated and usable end-to-end.
Forms	Offer, listing, payment, review	Client-side validation plus server error display.
Routing	Public + protected customer + protected agent routes	Direct URL access is role-gated.

10. Integration Workstream

Integration is not a final-week activity. The source plan assigns this owner from the beginning to maintain the contract, environment, CORS, smoke tests and integration health. `fileciteÖturn0file0ÖL206-L230Ö`

10.1 API contract format

API_CONTRACT.md
 For every endpoint define:

- Method + path
- Authentication requirement
- Allowed role
- Query/path params
- Request JSON
- Success response JSON
- Status codes
- Error response shape
- Database tables touched

10.2 Environment contract

Backend .env
 DATABASE_URL=...
 JWT_SECRET=...
 PORT=5000

Frontend .env
 VITE_API_URL=http://localhost:5000/api

10.3 Standard request lifecycle

User action

- > React component
- > API service / Axios
- > Express route
- > JWT + role middleware
- > Validation
- > Controller
- > Service
- > Repository / SQL

- > PostgreSQL
- > JSON response
- > React state/UI update

11. Critical End-to-End Flows

Flow	Sequence	What must be tested
Register → Login	Register PERSON + CUSTOMER/AGENT → commit → login → JWT	Duplicate email, wrong password, role claim, protected route.
Browse → Offer	Browse filters → listing detail → POST offer	Customer-only rule, listing exists, amount validation, UI confirmation.
Offer → Transaction	Agent accepts offer → DB transaction creates TRANSACTION + SALE/RENTAL subtype	Atomic commit/rollback, exclusive subtype, listing status update.
Transaction → Payment	Payment form → POST payment → history refresh	Transaction exists, amount > 0, unique reference.
Review	Customer posts review → listing review list refresh	Rating 1-5, ownership/purchase policy if imposed, duplicate review policy.

12. Team Division

Member	Primary ownership	Secondary responsibility	Must deliver
Member 1 — Frontend	React/Vite/Tailwind, routes, reusable UI, customer + agent screens	Mock API integration and frontend validation	All pages functional against mock responses; consistent UI system; loading/empty/error states.
Member 2 — Backend + DB	PostgreSQL, schema, seed, normalization, SQL, Express, auth, repositories/controllers	Postman collection and DBMS demo artifacts	Runnable DB + API, tested endpoints, constraints, one transaction-wrapped write.
Member 3 — Integration	API contract, Axios wiring, CORS/env, auth flow, E2E smoke tests	Report sequence diagrams + deployment setup	Real frontend ↔ backend flows with consistent error handling.

13. Dependency & Parallel Work Matrix

Task	Frontend	Backend/DB	Integration
Finalize schema + keys	Input only	LEAD	Review
API contract	Review + mock against it	Implement against it	LEAD / maintain
Static UI	LEAD	Independent	Observe
Seed database	No dependency	LEAD	Verify with smoke queries
Auth API + UI	Build UI/mock	Build API	Wire + test
Property/listing	Build screens	Build queries/endpoints	Connect once endpoint stable
Offers	Build form/dashboard	Build endpoints + DB rules	Connect + E2E
Transactions/payments	Build UI	Build atomic workflow	Connect + E2E
Reviews	Build UI	Build endpoints	Connect + E2E
Advanced DB	Surface results in UI if useful	LEAD	Validate outputs
Final testing	UI + smoke	Postman + SQL	E2E LEAD

14. Four-Week Suggested Timeline

Week	Frontend	Backend + DB	Integration
1	Vite/Tailwind scaffold; static pages; mock data	Finalize DDL, keys, constraints, seed; implement auth endpoints	Draft + freeze API contract; environment/CORS setup
2	Wire auth, browse, detail; filter UI	Properties, property types, ownership, listings, offers; first view	First real integration pass on auth + browse
3	Customer/agent dashboards; offer/payment/review forms	Transactions, subtype enforcement, payments, reviews; trigger(s)	Integrate remaining endpoints; contract-drift fixes; E2E flows
4	Polish UI, validation, loading/error/empty states	Indexes, advanced SQL, Postman collection, EXPLAIN ANALYZE	Full E2E regression; sequence diagrams; final demo script

15. Day-1 Shared Repository

```
real-estate-system/  
  frontend/  
  backend/  
  db/  
  docs/  
  API_CONTRACT.md  
  UI_DESIGN_SYSTEM.md  
  README.md  
  .env.example  
  .gitignore
```

The source plan specifically calls for db/schema.sql, db/seed.sql, API_CONTRACT.md/openapi.yaml, .env.example, README.md and a UI design-system document to be committed before feature branches multiply.

fileciteÖturn0file0ÖL231-L238Ö

16. Git Workflow

```
main          # protected, demo-ready  
develop       # integration branch  
feat/frontend/*  
feat/backend/*  
feat/db/*  
feat/integration/*
```

- One feature = one branch; small commits with clear messages.
- Schema changes and API contract changes require review/approval from all three members.
- Never commit .env secrets; commit .env.example.
- Merge to develop first; main only after integration smoke test passes.

17. Advanced DB Upgrade Roadmap

Feature	When	Why it matters	Demo
Indexes	After stable browse queries	Faster filtering/sorting/join lookups	EXPLAIN ANALYZE before/after
View	MVP/Week 2-4	Simplifies non-trivial	SELECT from

		browse/report join	active_listings_full
Trigger	Week 3-4	Enforces rules at DB layer	Exclusive transaction subtype / listing close
Stored function	Advanced phase	Encapsulates reusable DB logic	Function called from API
Materialized view	Advanced phase	Precomputes expensive reporting results	Refresh + timing comparison
Full-text search	Advanced phase	Better property-description search	Search relevance demo
PostGIS	Optional advanced phase	Nearby-property/geospatial queries	Properties within radius
Partitioning	Only if dataset grows	Demonstrate scale-aware design	Partition strategy + query plan

18. What to Show in the DBMS Viva/Demo

- EER → cardinalities → relational mapping walkthrough.
- PK/FK mapping for every relation, especially the N:N OWNERSHIP junction and ISA subtype PK/FK mapping.
- Individual normalization: identify candidate keys, functional dependencies, partial/transitive dependencies, then show legitimate decomposition/verification up to BCNF.
- Demonstrate one view, one trigger, one multi-statement SQL transaction, and a few non-trivial joined/aggregate queries.
- Show EXPLAIN ANALYZE before/after an index.
- Show a complete end-to-end flow: register → login → browse → offer → transaction → payment → review.
- Be able to explain why SQL is written manually and where each query lives in the repository layer.

19. Final Definition of Done

Area	Complete when...
Database	All relations created; PK/FK/check/unique rules pass; seed works on a fresh DB; normalization documented; queries return expected results.
Backend	Server starts from README instructions; auth works; every agreed endpoint tested in Postman; no unparameterized user input in SQL; errors have one shape.
Frontend	All required pages route correctly; forms validate; loading/empty/error states exist; customer vs agent UI is appropriately gated.
Integration	Frontend uses the real backend; JWT works; CORS/env configured; all 5 critical flows pass end-to-end.
Advanced	At least one view, one trigger, one transactional workflow, indexes + EXPLAIN ANALYZE demonstrated.
Team handoff	Any member can clone the repository, create DB, seed it, start backend/frontend and run the demo without hidden setup steps.

20. START HERE — First 10 Actions

1. Freeze the 14 relationship cardinalities exactly as the source-of-truth list.
2. Freeze the 13 relations, exact columns, PKs and FKs in schema.sql/API documentation.

3. Confirm the final subtype rule: TransactionType = SALE or RENTAL and never both.
4. Finalize data types, NOT NULL, UNIQUE, CHECK and delete/update behavior.
5. Create the shared repository skeleton and .env.example.
6. Write API_CONTRACT.md before backend/frontend feature work diverges.
7. Backend member creates schema.sql + constraints.sql + seed.sql and proves a clean rebuild from scratch.
8. Frontend member scaffolds React/Vite/Tailwind and builds static pages against mock responses.
9. Integration member sets up Axios, CORS, env wiring, JWT contract and the first auth smoke test.
10. Run the first shared checkpoint: fresh clone → DB seed → backend → frontend → login → browse, with all three members validating the contracts.

Source note: This expanded document is based on the uploaded 7-page build plan and preserves its core architectural decisions while adding the requested cardinality/key-mapping detail. The original plan recommends raw SQL with pg, contract freezing before parallel work, the 13-entity backend/database scope, the three workstreams, a 4-week cadence, and DBMS-specific demo artifacts. [fileciteÖturn0file0ÖL2-L64Ô](#) [fileciteÖturn0file0ÖL134-L175Ô](#) [fileciteÖturn0file0ÖL206-L230Ô](#)

APPENDIX — 3-PERSON WORK DIVISION

This section is the final team execution plan. The project is divided into three clear workstreams: **Frontend**, **Backend + Database**, and **Frontend-Backend Integration**. The goal is to allow parallel development while keeping the database schema and API contract as shared sources of truth.

Team Ownership at a Glance

Person	Primary Ownership	Core Deliverables
Person 1 — Frontend	React/Vite/Tailwind UI and client-side experience.	Pages, reusable components, routing, dashboards, forms, filters, frontend validation, loading/error/empty states, responsive UI.
Person 2 — Backend + Database	PostgreSQL schema and Node/Express server.	DDL, PK/FK constraints, seed data, normalization documentation, SQL queries, repositories, controllers, REST APIs, authentication, authorization, transactions, DB features.
Person 3 — Integration	The contract and seam between frontend and backend.	API contract, Axios integration, JWT flow, CORS/env setup, response/error mapping, end-to-end workflows, smoke tests, integration debugging, final demo readiness.

Person 1 — FRONTEND

Mission: Build the complete user-facing React application and make every screen usable against mock API responses first, then the real API.

Phase-by-Phase Tasks

Phase	Tasks	Deliverable / Done When
1. Setup	Create Vite React app; configure Tailwind; React Router; Axios instance; base folder structure; shared design tokens.	Frontend runs locally with agreed commands and shared UI foundation.
2. Auth UI	Login, registration, role selection, validation, auth context, protected-route placeholders.	Customer/agent can navigate through auth screens using mock responses.
3. Property discovery	Home, browse listings, property cards, search, filters, sorting, pagination UI, property details.	User can browse and inspect realistic property/listing data.
4. Customer flows	Customer dashboard, owned properties, offers, transactions, payments, reviews, profile.	All customer journeys have complete screens and form states.
5. Agent flows	Agent dashboard, managed listings, add/edit listing, offer review, transaction/payment actions.	Agent can navigate and perform every UI action expected by the API contract.
6. Quality	Responsive layouts, loading skeletons, empty states, validation, error messages, confirmation modals, accessibility basics.	Every API-driven screen has loading/error/empty/success states.

Frontend must receive from the other members: finalized field names, role names, API endpoints, request/response JSON shapes, status codes, and authentication behavior.

Person 2 — BACKEND + DATABASE

Mission: Own the PostgreSQL database and the Express REST API. This member protects the relational design and ensures that application behavior is enforced at the server/database layer rather than only in the UI.

Phase-by-Phase Tasks

Phase	Tasks	Deliverable / Done When
1. Schema lock	Implement the agreed 13 relations; define data types, PKs, FKs, NULL/NOT NULL rules, UNIQUE constraints, CHECK constraints, delete/update behavior.	schema.sql runs on a clean PostgreSQL database without manual fixes.
2. Data + normalization	Create realistic seed data; document functional dependencies and individual 1NF→2NF→3NF→BCNF analysis; verify composite OWNERSHIP key and subtype mapping.	Seeded DB is internally consistent and normalization report matches the implemented schema.
3. SQL layer	Write parameterized queries for CRUD, joins, filters, aggregates, subqueries/CTEs where useful; add repository/DAO files.	All core DB operations are available without SQL embedded in route files.
4. Express API	Routes, controllers, services/repositories, validation, standardized errors, pagination/filtering.	Postman can exercise every agreed endpoint.
5. Auth/security	Registration transaction, bcrypt hashing, login, JWT creation/verification, role guards, authorization checks.	Customer/agent permissions are enforced by backend.
6. Multi-table transactions	Use BEGIN/COMMIT/ROLLBACK for registration and transaction/subtype creation or other atomic workflows.	Failure in any dependent write rolls back the complete operation.
7. Advanced DB	Views, triggers, indexes, EXPLAIN ANALYZE and other approved advanced features.	At least one useful view, trigger, index comparison and non-trivial query are demonstrable.

Backend must receive from the other members: finalized UI requirements and API consumer needs; however, database/schema and API contract changes require agreement from all three members.

Person 3 — FRONTEND–BACKEND INTEGRATION

Mission: Integration is not a final-week task. This member owns the contract between the frontend and backend from Day 1 and continuously verifies that both sides remain compatible.

Phase-by-Phase Tasks

Phase	Tasks	Deliverable / Done When
1. Contract	Create API_CONTRACT.md/OpenAPI; freeze endpoint paths, methods, payloads, response shapes, status codes, auth requirements and common error format.	Frontend can mock the API and backend can implement against exactly the same contract.
2. Environment	Configure backend env variables, Vite API URL, CORS, optional dev proxy, database startup/seed commands.	Fresh clone can run frontend + backend + DB using documented setup.
3. Auth integration	Connect login/register to real API; handle JWT; Axios interceptor; protected routes; 401/403 behavior; logout.	Real authentication works from browser through Express to PostgreSQL.
4. Feature integration	Connect browse/search, property details, listing management, offers, transactions, payments and reviews one flow at a time.	Each feature works end-to-end against real data.
5. Data/error mapping	Normalize backend responses for UI; map 400/401/403/404/409/422/500 into user-friendly states; prevent inconsistent response handling.	No page has ad-hoc API parsing or unexplained errors.
6. E2E testing	Run customer and agent workflows; verify DB changes after UI actions; maintain smoke-test checklist/Postman collection.	Critical workflows pass from UI → API → DB → API → UI.
7. Final integration	Resolve contract drift, merge branches, verify environment setup, capture demo screenshots, prepare sequence diagrams and demo script.	Single reproducible integrated build is ready for submission/demo.

Shared Rules — All 3 Members

1. **Schema is frozen:** nobody independently renames tables/columns or changes PK/FK structure.
2. **API contract is frozen:** frontend and backend changes must update the shared contract before implementation diverges.
3. **Integration starts immediately:** use mocks while the backend is being built; do not wait until the entire project is finished.
4. **Git discipline:** feature branches, small commits, pull requests, and review of shared-contract changes.
5. **Definition of Done:** a feature is not complete until its happy path, validation, loading state, empty state, error state, authorization behavior, API test and integration smoke test are covered.

Critical Dependency Chain

Database/EER decisions → Relational schema → API contract → Frontend mocks + Backend implementation → Real integration → E2E testing → Advanced DB features → Final demo.